Real-world (a cold design-review interview on your finished capstone) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `capstone_review.md`: this is the final exam of the whole curriculum — a genuinely cold review of your finished capstone, jumping unpredictably between cost, testing, database design, and auth, at a real reviewer's pace, not yours. **If not:** the very first time you faced this kind of rapid cross-examination would be a final-round job interview, with a real offer on the line.

Write your own version first, in `practice/capstone_review.md`, then compare.

Model interview for: "Interview me about my capstone" — this solution uses the same ContentForge / Project 5 system from Docs 16 and 17 as the running example, extended to the full production system (RAG, database, auth, deployment, cost) this document actually asks you to build. (In this example, the capstone design added a fact-checker agent to Project 4's team, to check each draft's claims against the RAG sources.)

Solution

Story: this ties every answer back to a specific requirement or failure mode it addresses, including volunteering the genuine gaps yourself before being asked. **If not:** your answers would just describe the pieces you built without ever saying *why* each one is actually there.

Reviewer: "Same as before, but I'm not going in system order — I'll jump around."

Reviewer, opening on cost, not architecture: "Your requirements said $200/month. Walk me through what's actually keeping you under that."

"Three things enforce it: a per-request token budget on the generation calls, the retrieval cache in front of the vector store that avoids re-embedding repeated queries, and a monthly spend alert wired to the LLM provider's usage API that pages me before the account gets close to the cap. The cache is the biggest lever — most of the cost at this traffic volume is repeated similar queries, not unique ones."

Reviewer, jumping to testing: "What's actually tested versus what you're hoping works?"

"Each agent's node function has unit tests against fixed inputs — the fact-checker's grounding logic, specifically, is tested against cases where the source material does and doesn't support a claim, since that's the highest-stakes piece. The supervisor's routing logic has integration tests that run a

full simulated request through 2-3 known paths, including the backward loop from fact-check failure to the writer. What's NOT tested end-to-end: the actual deployed API under real concurrent load — I've load-tested it manually once, not as part of CI, which is a real gap for a system with a stated 50,000 requests/day target."

Reviewer, back to database, from a different angle than before: "You said Postgres and the vector store are separate because of different access patterns. What happens to a request if the vector store is briefly unavailable but Postgres is fine?"

"That's handled as a degraded-mode path, not a full failure — the agent's RAG tool call is wrapped the way Doc14 trains, so a vector-store timeout doesn't crash the request. It falls back to answering from the model's general knowledge with an explicit disclaimer in the output that retrieval was unavailable, rather than either crashing or silently answering as if grounding had succeeded."

Why this level works: each answer is specific to the actual requirement or actual failure mode, not a generic description of the piece. The testing answer volunteers the real gap (no automated load test) unprompted, which is what Doc17's material already established as a stronger signal than waiting to be caught. The database-availability answer shows the design was thought through past the happy path, tying back to Doc14's failure-handling vocabulary.